



CSE3433 SOFTWARE ARCHITECTURE

PROJECT TITLE:

CLOUD NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

GROUP 4

Name	Matric No.
EASHENRAJ A/L NAGARAJAN	S76773
MOHAMMAD ADAM BASHEER B. MOHD AZRUL	S75710
AMMAR HAZIQ B. SAIFUL BAHRI	S75772

PREPARED FOR:

DR MOHAMAD NOR HASSAN

Introduction

This report outlines the deployment architecture and user interface implementation for the Cloud Native Learning Management System (Cloud LMS). The platform is designed to provide dedicated, scalable workspaces for both students and instructors, hosted entirely on cloud infrastructure.

1. Cloud Deployment & Architecture

The project utilizes a modern cloud-native layered architecture, separating the frontend, backend, and database into distinct cloud services:

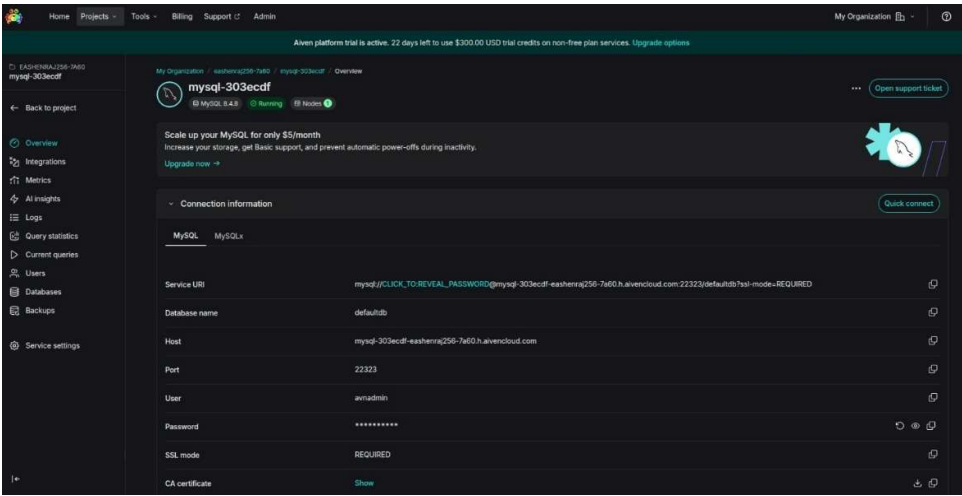


Figure 1: Aiven MySQL for backend

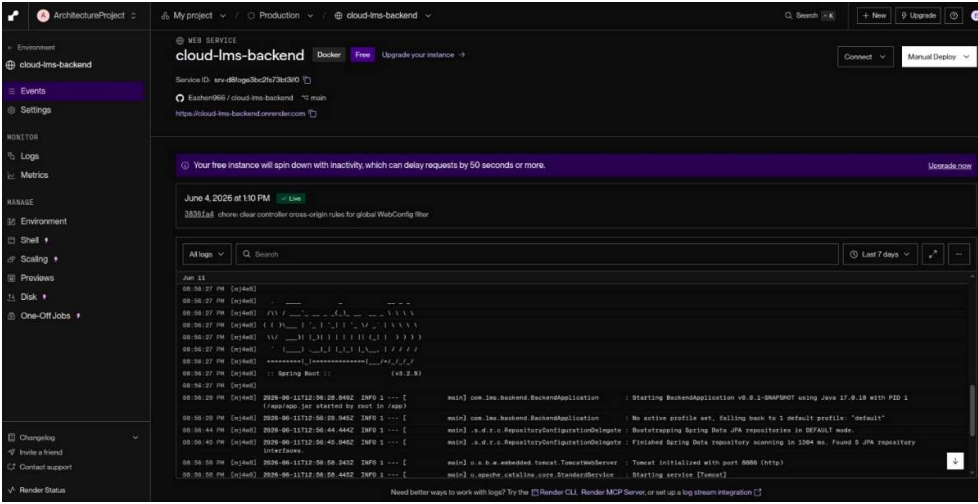


Figure 2: Cloud LMS backend

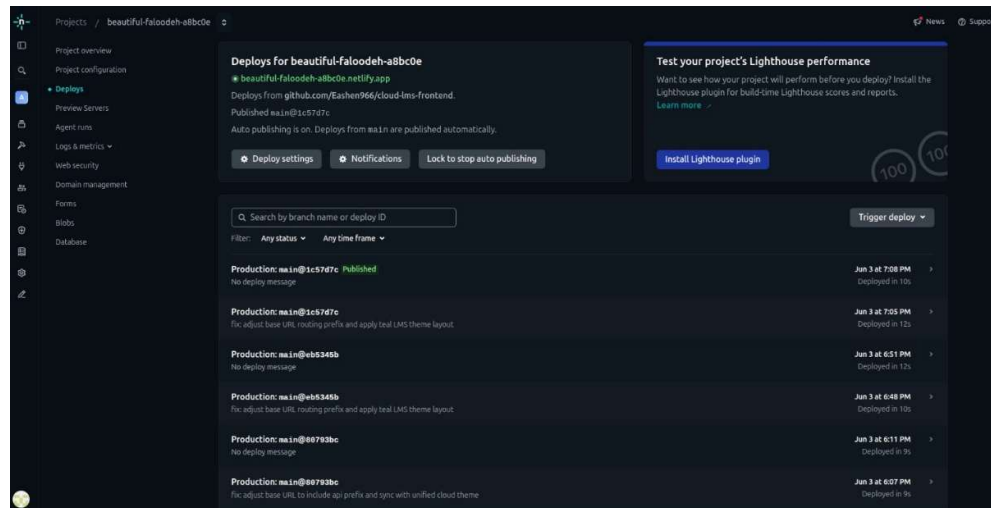


Figure 3: Netlify frontend deployment

2. System User Interface

The Cloud LMS feature a clean and simple user interface with role-based access control, ensuring different views for each student and staffs.

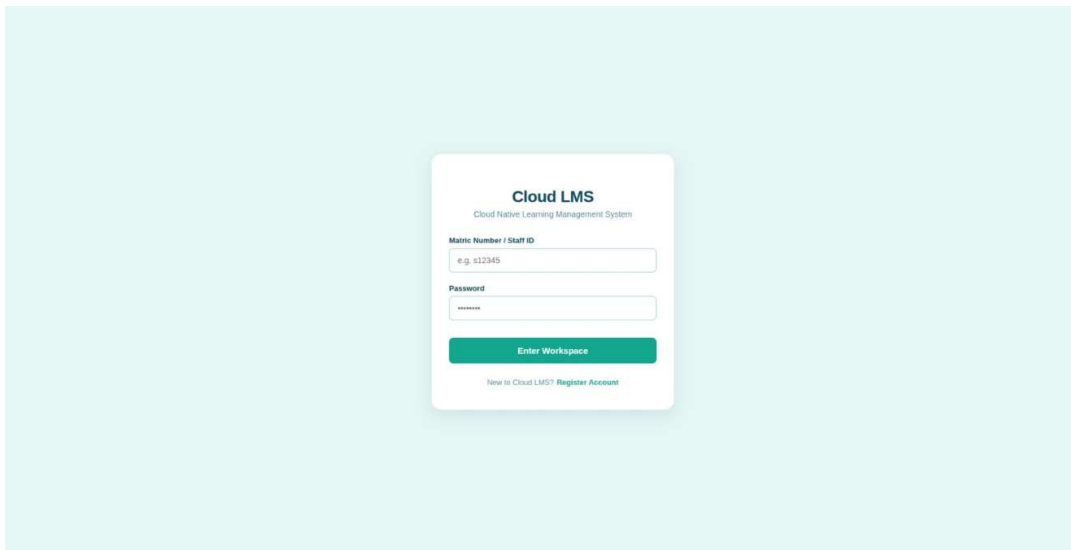
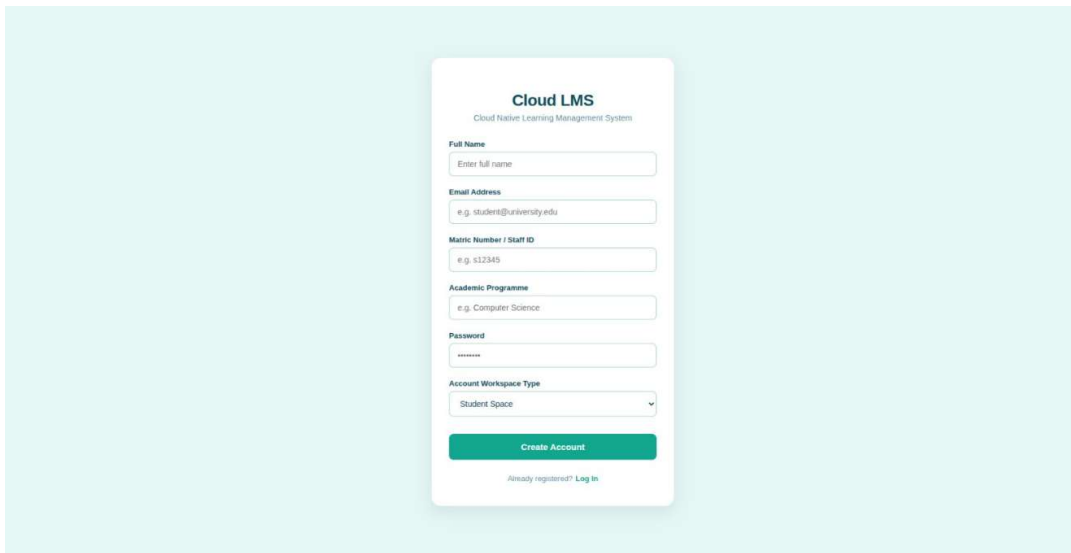


Figure 4: Login User Interface

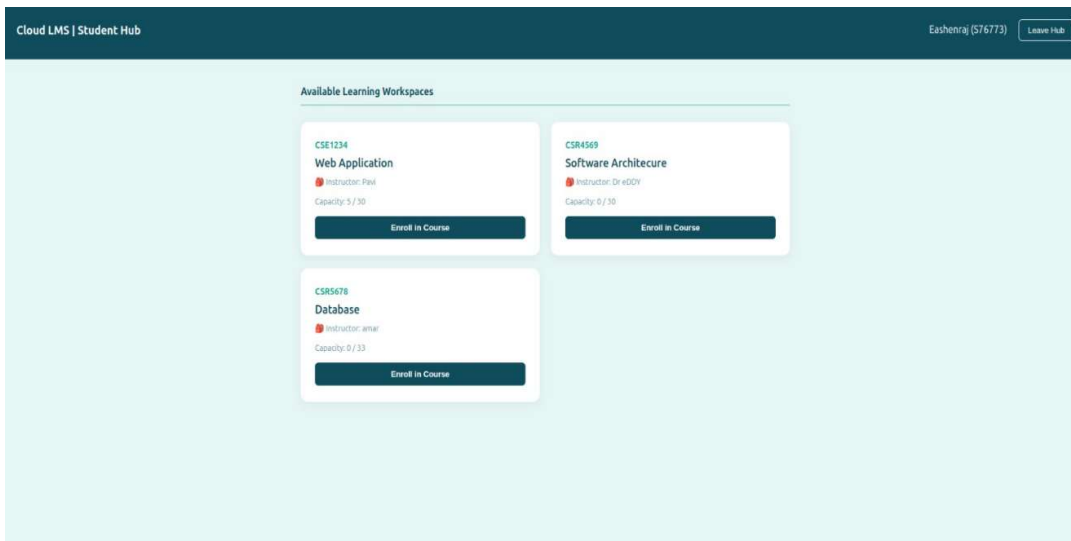
Figure 0 shows the login interface of the Cloud LMS. Users can access the platform through login by entering their matric number (for students) or staff ID (for instructors) with their registered passwords.



The image shows a registration form for a Cloud LMS. The form is titled "Cloud LMS" with the subtitle "Cloud Native Learning Management System". It contains several input fields: "Full Name" (placeholder: "Enter full name"), "Email Address" (placeholder: "e.g. student@university.edu"), "Matric Number / Staff ID" (placeholder: "e.g. s12345"), "Academic Programme" (placeholder: "e.g. Computer Science"), "Password" (placeholder: "*****"), and a dropdown menu for "Account Workspace Type" (selected: "Student Space"). A green "Create Account" button is at the bottom, with a link "Already registered? Log In" below it.

Figure 5: Registration User Interface

Figure 0 displays user interface for the registration form. The users to register their account into the system. They can register by entering their full name, email, academic programme and account workspace type.



The image shows a student dashboard for a Cloud LMS. The header is "Cloud LMS | Student Hub" with a user profile "Eashenra (S76773)" and a "Learn Hub" button. The main content is titled "Available Learning Workspaces" and displays three course cards. Each card shows a course code, name, instructor, and capacity, with an "Enroll in Course" button.

Course Code	Course Name	Instructor	Capacity
CSE1234	Web Application	Instructor: Paul	Capacity: 5 / 30
CSR4569	Software Architecture	Instructor: Dr. eSDN	Capacity: 0 / 30
CSR5678	Database	Instructor: amai	Capacity: 0 / 33

Figure 6: Student Dashboard Interface

Figure 0 is the dashboard of the system that display all the available courses for students to enrol. Each course will provide information such as course code, course name, and capacity.

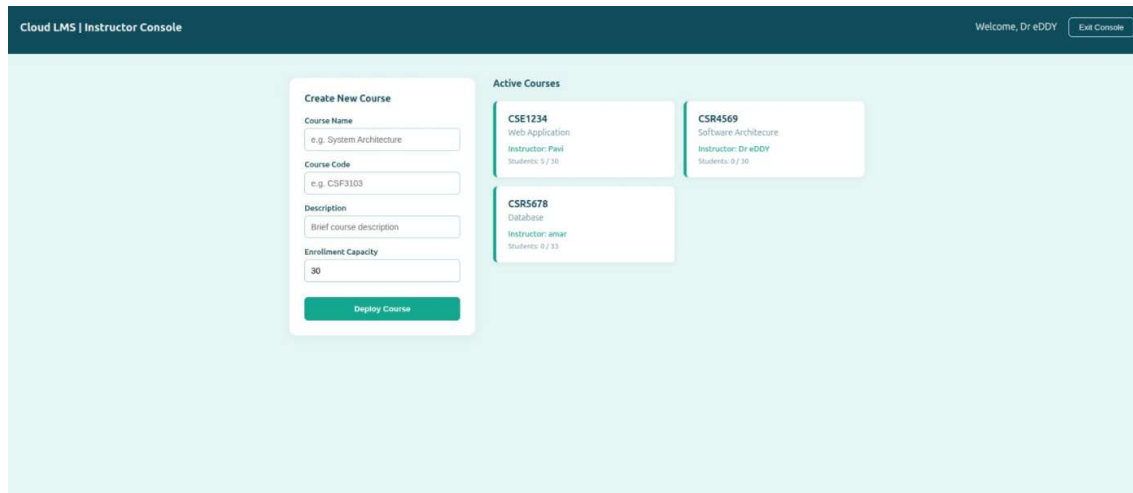


Figure 7: Instructor Dashboard Interface

Figure 0 show the interface where the instructor uses to create a new course. They can input the essential course details such as course name, course code, description and the enrolment capacity of students. Created course will be displayed to the student in the student dashboard.

3. Codebase Structure and Architectural Patterns

The Cloud LMS project is organized into distinct backend and frontend directories, accompanied by a docker file to orchestrate the contained services.

Backend Implementation

- The backend is built as a Java application utilizing Maven.
- The source code strictly adheres to a Layered Architecture design pattern, organizing classes into controller, model, repository, and service packages.
- The code follows the Layered Architecture design pattern which organizes the layers into controller, model, repository and service packages.
- This layered structure manages core system entities, with dedicated files for assignment, course content, assignment submission, and user across each architectural layer.
- A Dockerfile is also included directly within the backend directory to facilitate the containerization of the Spring Boot service.

Frontend Implementation

- The frontend is a modern web application developed using React and Vite
- The user interface is driven by distinct views which contains InstructorDashboard.jsx, Login.jsx, and StudentDashboard.jsx.
- The frontend architecture promotes modularity by separating reusable UI elements into a components folder and API communication logic into a services folder.